

Recherche Tabou

Algorithme

Étapes :

1. **Initialiser** une solution x_0 , liste taboue vide T , paramètres (M, k)
 2. **Tant que** critère d'arrêt non atteint :
 - Générer voisinage $V(x_c)$
 - **Choisir** le meilleur $x' \in V(x_c)$ **non-tabou** (ou avec aspiration)
 - $x_c \leftarrow x'$
 - **Mettre à jour** T (ajouter mouvement inverse, FIFO ou temps k)
 - (Optionnel) Mettre à jour mémoire long terme
 3. **Retourner** meilleure solution trouvée
-

Formules clés

Fitness QAP :

$$f = \sum_{i,j} f_{ij} \cdot d_{r_i r_j}$$

Évaluation voisin :

$$\Delta = f(x') - f(x)$$

Condition tabou (exemple) : mouvement (r, s) interdit si

$$T_{i_s r} > t \quad \text{et} \quad T_{i_r s} > t$$

où T est la matrice tabou, t l'itération actuelle.

Caractéristiques

Espace de recherche : Combinatoire (permutations, graphes, etc.)

Convergence : Conditionnelle (si espace fini, voisinage symétrique, accessibilité), mais temps exponentiel possible.

Mémoire : Court terme (liste taboue FIFO/tenure k) + long terme (fréquences des mouvements).

Aspiration : Outrepasser tabou si solution meilleure que toutes précédentes.

Paramètres de guidage

- **Taille liste taboue** (M) : contrôle mémoire court terme
- **Temps de bannissement** (k) : durée d'interdiction d'un mouvement
- **Structure du voisinage** : type et nombre de mouvements
- **Critère d'aspiration** : condition pour ignorer tabou

Recuit Simulé (Simulated Annealing)

Algorithme

Recuit Simulé Standard

1. Initialisation

- Choisir une solution initiale x_{current} .
- Choisir une température initiale élevée T_0 .
- Définir un planning de refroidissement et un critère d'arrêt.

2. Boucle principale (jusqu'à ce que le critère d'arrêt soit satisfait)

- Pour un nombre donné d'itérations à cette température :
 - Générer une solution voisine x_{new} à partir de x_{current} (mouvement élémentaire).
 - Calculer $\Delta E = E(x_{\text{new}}) - E(x_{\text{current}})$.
 - Si $\Delta E \leq 0$, accepter x_{new} ($x_{\text{current}} \leftarrow x_{\text{new}}$).
 - Sinon, accepter x_{new} avec probabilité $p = \exp(-\Delta E/T)$ (Règle de Metropolis).
- Diminuer la température T selon le planning de refroidissement.

3. Résultat

- Retourner la meilleure solution rencontrée.

Recuit Parallèle (Parallel Tempering)

1. Initialisation

- Lancer M replicas du système avec des températures fixes $T_1 < T_2 < \dots < T_M$.
- Initialiser une configuration aléatoire pour chaque replica.

2. Boucle principale

- Pour chaque replica i de manière (quasi-)indépendante :
 - Effectuer des étapes de Recuit Simulé standard à température constante T_i .
- Périodiquement, proposer un échange de configurations entre deux replicas adjacents en température (i et $i + 1$).
- Accepter l'échange avec probabilité $p = \min(1, e^{-\Delta_{ij}})$ où $\Delta_{ij} = (E_i - E_j)(1/T_j - 1/T_i)$.

Formules Mathématiques Essentielles

Règle d'Acceptation de Metropolis

Probabilité d'accepter une solution plus mauvaise :

$$p_{\text{accept}} = \min\left(1, \exp\left(-\frac{\Delta E}{T}\right)\right)$$

avec $\Delta E = E_{\text{new}} - E_{\text{old}}$.

Planning de Refroidissement Typique (Géométrique)

$$T_{k+1} = \alpha \cdot T_k$$

avec $\alpha \in]0, 1[$, souvent $\alpha = 0.9$.

Caractéristiques

Espace de Recherche et Voisinage

- **Espace** : Discrét ou continu. Représentation des solutions cruciale.
- **Mouvements** : Doivent être réversibles et connecter l'espace. Choix impactant fortement la performance (ex. 2-opt pour TSP).

Convergence

- **Convergence théorique** : Sous conditions (température initiale assez haute, refroidissement assez lent $T(t) \propto C / \log(t)$), convergence en probabilité vers l'optimum global.
- **Pratique** : La convergence garantie est souvent trop lente. Les paramètres sont réglés pour un compromis temps/qualité.

Paramètres Guidés (Guiding Parameters)

1. **Température Initiale T_0**
 - Doit permettre une exploration large.
 - Heuristique : T_0 tel que $\exp(-\langle |\Delta E| \rangle / T_0) = \tau_0$, avec $\tau_0 \approx 0.5$.
2. **Loi de Refroidissement**
 - Vitesse de décroissance (α).
 - Nombre d'itérations par palier de température.
3. **Critère d'Arrêt**
 - Température minimale $T_{\min}$.
 - Nombre maximum d'itérations.
 - Absence d'amélioration après plusieurs paliers.
4. **(Pour Recuit Parallèle)**
 - Nombre de replicas M (souvent $M \approx \sqrt{N}$).
 - Séquence des températures $\{T_i\}$.
 - Fréquence des échanges.

Informations Clés

- **Métaheuristique** : Inspirée de la physique (recuit métallurgique).
- **Gestion Exploration/Exploitation** : Contrôlée par la température T .
- **Avantage Principal** : Capacité à échapper aux minima locaux en acceptant des dégradations temporaires.
- **Inconvénient Principal** : Nombreux paramètres à régler, performance dépendante de ce réglage.
- **Variante Majeure** : *Parallel Tempering* pour améliorer l'exploration et permettre le parallélisme.

Algorithme

Standard

1. Initialiser solution x , température T
2. Tant que critère non atteint :
 - Générer voisin x'
 - $\Delta E = E(x') - E(x)$
 - Si $\Delta E \leq 0 : x \leftarrow x'$
 - Sinon : $x \leftarrow x'$ avec proba $e^{-\Delta E/T}$
 - Diminuer T selon planning

Parallèle

1. Lancer M replicas à températures fixes $T_1 < \dots < T_M$
2. Parallèlement :
 - Chaque replica évolue indépendamment
 - Échanges périodiques entre replicas adjacents selon $p = \min(1, e^{-\Delta_{ij}})$

Formules

$$p_{\text{accept}} = \min(1, e^{-\Delta E/T})$$

$$T_{k+1} = \alpha T_k \quad (0 < \alpha < 1)$$

$$\Delta_{ij} = (E_i - E_j)(1/T_j - 1/T_i)$$

Caractéristiques

Espace : Continu ou discret

Convergence : Probabiliste vers optimum global si $T(t) \propto C/\log(t)$

Paramètres :

- T_0 (température initiale)
- Planning refroidissement (α , itérations/palier)
- Critère d'arrêt
- Mouvements élémentaires

Optimisation par Colonie de Fourmis (ACO)

Algorithme

Étapes principales de l'Ant System (AS) :

1. **Initialisation** : Déposer une petite quantité de phéromone $\tau_{ij}(0)$ sur chaque arête.
2. **Construction des solutions** : Pour chaque fourmi k :
 - Construire une solution complète par séquences de choix probabilistes.

- La probabilité de choisir l'élément j depuis l'état i dépend de la phéromone τ_{ij} et d'une heuristique η_{ij} .
3. **Évaluation** : Calculer la qualité $f(s_k)$ de chaque solution construite.
 4. **Mise à jour des phéromones** :
 - **Évaporation** : Réduire toutes les phéromones par un facteur $(1 - \rho)$.
 - **Dépôt** : Ajouter de la phéromone sur les composants des bonnes solutions.
 5. **Itération** : Répéter les étapes 2-4 jusqu'à critère d'arrêt.

Formules Mathématiques Essentielles

Règle de décision probabiliste (AS) :

$$p_{ij}^k = \frac{[\tau_{ij}]^\alpha \cdot [\eta_{ij}]^\beta}{\sum_{l \in N_i^k} [\tau_{il}]^\alpha \cdot [\eta_{il}]^\beta}$$

Mise à jour des phéromones (AS) :

$$\tau_{ij} \leftarrow (1 - \rho)\tau_{ij} + \sum_{k=1}^m \Delta\tau_{ij}^k$$

avec $\Delta\tau_{ij}^k = \frac{Q}{L_k}$ si l'arête (i, j) est dans le tour de la fourmi k .

Règle de transition pseudo-aléatoire (ACS) :

$$j = \begin{cases} \arg \max_{l \in N_i^k} \{[\tau_{il}] \cdot [\eta_{il}]^\beta\} & \text{si } q \leq q_0 \\ \text{règle proportionnelle de l'AS} & \text{sinon} \end{cases}$$

Caractéristiques

Espace de Recherche

- Problèmes **combinatoires** (TSP, QAP, ordonnancement).
- Solutions construites **incrémentalement**.
- Espace exploré via **mémoire collective** (phéromones).

Convergence

- Convergence **asymptotique** vers l'optimum garanti avec paramètres adaptés.
- **Équilibre exploration/exploitation** crucial.
- Risque de **stagnation/prématurité** si mauvais réglage.

Paramètres Guidés (Guided Parameters)

Ant System (AS)

- $\alpha \geq 0$: Poids de la phéromone (exploitation)
- $\beta \geq 0$: Poids de l'heuristique (exploration)
- $\rho \in [0,1]$: Taux d'évaporation (oubli)
- $Q > 0$: Constante de dépôt (échelle phéromonale)

- m : Nombre de fourmis (parallélisme)

Ant Colony System (ACS)

- $q_0 \in [0,1]$: Seuil de décision déterministe
- $\phi \in [0,1]$: Taux d'évaporation locale

Valeurs typiques (TSP) :

$$\alpha = 1, \quad \beta = 5, \quad \rho = 0.5, \quad m = n, \quad Q = 1$$

Informations Clés

Mécanismes Fondamentaux

- **Stigmergie** : Coordination indirecte via modification environnement (phéromones).
- **Auto-organisation** : Ordre global émerge d'interactions locales simples.
- **Boucle de rétroaction positive** : Renforcement des bonnes solutions.

Avantages

- **Robuste** : Tolère perte d'agents/erreurs.
- **Parallélisable** : Fourmis indépendantes.
- **Flexible** : Adaptable à divers problèmes.
- **Équilibré** : Gère naturellement exploration/exploitation.

Limites

- **Nombreux paramètres** à régler.
- **Convergence lente** sur grands problèmes.
- **Performance** dépend fortement de l'heuristique choisie.
- **Stagnation** possible (tous les agents suivent même chemin).

Variantes Principales

1. **Ant System (AS)** : Version originale.
2. **Ant Colony System (ACS)** : Avec règle pseudo-aléatoire et mise à jour élitiste.
3. **MAX-MIN Ant System** : Bornes sur les phéromones pour éviter stagnation.

Domaines d'Application

- Routage réseaux
- Ordonnancement
- Classification données
- Apprentissage automatique
- Robotique en essaim

Algorithme

AS (Ant System) :

1. Initialiser les phéromones
2. Chaque fourmi construit une solution via choix probabilistes basés sur phéromones τ et heuristique η
3. Évaluer les solutions
4. Évaporer les phéromones : $\tau \leftarrow (1 - \rho)\tau$
5. Déposer des phéromones sur les bonnes solutions
6. Répéter 2-5

ACS (Ant Colony System) différences :

- Règle de décision : avec proba q_0 , choix déterministe ($\max [\tau][\eta]^\beta$) ; sinon règle AS
- Mise à jour locale immédiate de l'évaporation après chaque choix
- Seule la meilleure fourmi globale dépose des phéromones

Formules

Règle de décision AS :

$$p_{ij} = \frac{[\tau_{ij}]^\alpha [\eta_{ij}]^\beta}{\sum [\tau_{il}]^\alpha [\eta_{il}]^\beta}$$

Mise à jour phéromones AS :

$$\tau_{ij} \leftarrow (1 - \rho)\tau_{ij} + \sum_k \Delta\tau_{ij}^k$$

avec $\Delta\tau_{ij}^k = Q/L_k$ si arête dans solution k

Caractéristiques

Espace recherche : Combinatoire, construction incrémentale

Convergence : Asymptotique, risque de stagnation, équilibre exploration/exploitation critique

Paramètres guidés :

- AS : α (phéromone), β (heuristique), ρ (évaporation), m (#fourmis), Q
- ACS : $+ q_0$ (seuil déterministe), ϕ (évaporation locale)

Valeurs typiques TSP : $\alpha = 1$, $\beta = 5$, $\rho = 0.5$, $m = n$, $Q = 1$

Particle Swarm Optimization (PSO)

Algorithme

1. Initialiser un essaim de N particules avec positions aléatoires $\mathbf{x}_i(0)$ et vitesses nulles $\mathbf{v}_i(0)$ dans l'espace de recherche borné S .
2. Évaluer la fitness $f(\mathbf{x}_i(t))$ pour chaque particule.
3. Mettre à jour pour chaque particule :
 - Sa meilleure position personnelle $\mathbf{x}_i^{\text{best}}(t)$
 - La meilleure position globale $\mathbf{B}(t)$ de l'essaim
4. Mettre à jour la vitesse et la position de chaque particule selon les équations de mouvement.
5. Répéter les étapes 2-4 jusqu'à convergence ou nombre maximal d'itérations.

Formules mathématiques importantes

Mise à jour de la vitesse

$$\mathbf{v}_i(t+1) = \omega \mathbf{v}_i(t) + c_1 r_1 [\mathbf{x}_i^{\text{best}}(t) - \mathbf{x}_i(t)] + c_2 r_2 [\mathbf{B}(t) - \mathbf{x}_i(t)]$$

Mise à jour de la position

$$\mathbf{x}_i(t+1) = \mathbf{x}_i(t) + \mathbf{v}_i(t+1)$$

Meilleure position personnelle

$$\mathbf{x}_i^{\text{best}}(t) = \begin{cases} \mathbf{x}_i(t) & \text{si } f(\mathbf{x}_i(t)) \text{ meilleure que } f(\mathbf{x}_i^{\text{best}}(t-1)) \\ \mathbf{x}_i^{\text{best}}(t-1) & \text{sinon} \end{cases}$$

Meilleure position globale

$$\mathbf{B}(t) = \arg \max_{\mathbf{x}_i^{\text{best}}(t)} f(\mathbf{x}_i^{\text{best}}(t))$$

Caractéristiques

Espace de recherche

- Optimisation continue ($S \subset \mathbb{R}^n$)
- Extensions possibles pour problèmes discrets/combinatoires via méthodes d'arrondi

Convergence

- Convergence rapide vers un optimum (local ou global)
- Caractérisée par un équilibre exploration/exploitation

Guided parameters

- ω (**coefficient d'inertie**) : Contrôle la persistance de la vitesse précédente
 - Typiquement $\omega \in [0.4, 0.9]$

- Valeur courante : $\omega \approx 0.9$
- c_1 (**coefficient cognitif**) : Pondère l'attraction vers la meilleure position personnelle
 - Valeur courante : $c_1 \approx 2$
- c_2 (**coefficient social**) : Pondère l'attraction vers la meilleure position globale
 - Valeur courante : $c_2 \approx 2$
- r_1, r_2 : Nombres aléatoires uniformes dans $[0, 1[$ pour stochasticité
- $v_{\max}$: Limite de vitesse pour éviter l'explosion des valeurs

Informations clés

Principes fondamentaux

- Métaheuristique basée sur une population (essaim de particules)
- Inspirée du comportement collectif d'animaux sociaux (vol d'oiseaux, bancs de poissons)
- Chaque particule représente une solution candidate

Forces agissantes

1. **Inertie** : Persistance du mouvement précédent
2. **Attraction cognitive** : Vers la meilleure position personnelle (x_i^{best})
3. **Attraction sociale** : Vers la meilleure position globale (**B**)

Équilibre exploration/exploitation

- **Exploration** : Promue par l'inertie et la stochasticité
- **Exploitation** : Promue par les attractions cognitive et sociale

Avantages

- Simplicité d'implémentation
- Peu de paramètres à régler
- Convergence rapide
- Adaptable à divers types de problèmes

Limites

- Principalement conçu pour l'optimisation continue
- Risque de convergence prématurée vers un optimum local
- Performance sensible au réglage des paramètres

Variantes courantes

- PSO avec voisinage (social local au lieu de global)
- PSO avec constriction coefficient
- PSO pour optimisation discrète/combinatoire

i Particle Swarm Optimization (PSO)

Algorithme

1. Initialiser essaim aléatoire
2. Pour chaque particule, mettre à jour vitesse et position
3. Mettre à jour meilleures positions personnelle et globale
4. Répéter

Formules

$$\mathbf{v}_i = \omega \mathbf{v}_i + c_1 r_1 (\mathbf{x}_i^{\text{best}} - \mathbf{x}_i) + c_2 r_2 (\mathbf{B} - \mathbf{x}_i)$$

$$\mathbf{x}_i = \mathbf{x}_i + \mathbf{v}_i$$

Caractéristiques

- Espace : Continu (extensions discrètes)
- Convergence : Rapide
- Guided parameters : ω, c_1, c_2

Firefly Algorithm

Algorithme

1. Initialiser une population de n lucioles avec positions aléatoires $\mathbf{x}_i$.
2. Calculer l'intensité lumineuse $I_i = f(\mathbf{x}_i)$ pour chaque luciole.
3. Pour chaque paire de lucioles (i, j) où $I_i < I_j$:
 - Déplacer la luciole i vers j selon la règle de déplacement.
 - Mettre à jour l'intensité de i .
4. Trier les lucioles et mettre à jour la meilleure solution globale.
5. Répéter jusqu'à convergence.

Formules mathématiques importantes

Attractivité entre lucioles

$$\beta(r_{ij}) = \beta_0 \exp(-\gamma r_{ij}^2)$$

Règle de déplacement

$$\mathbf{x}_i^{\text{new}} = \mathbf{x}_i + \beta_0 \exp(-\gamma r_{ij}^2) (\mathbf{x}_j - \mathbf{x}_i) + \alpha \epsilon$$

Caractéristiques

Espace de recherche

- Optimisation continue principalement
- Versions discrètes existent

Convergence

- Convergence progressive vers les régions prometteuses
- Bon équilibre exploration/exploitation via γ

Guided parameters

- β_0 : Attractivité de base (typiquement 1)
- γ : Coefficient d'absorption lumineuse
 - Petit $\gamma \rightarrow$ exploration accrue
 - Grand $\gamma \rightarrow$ exploitation accrue
- α : Paramètre de randomisation (typiquement $\in [0, 1]$)
- ϵ : Bruit aléatoire (distribution uniforme, normale ou Lévy)

Informations clés

- Inspiré du comportement de bioluminescence des lucioles
 - Chaque luciole représente une solution candidate
 - L'intensité lumineuse correspond à la qualité de la solution
 - Les lucioles moins brillantes sont attirées par les plus brillantes
 - L'attractivité diminue exponentiellement avec la distance
 - Deux composantes de déplacement : attraction déterministe + bruit aléatoire
 - Adaptable à divers problèmes d'optimisation difficile
-

Cuckoo Search

Algorithme

1. Initialiser n nids avec solutions aléatoires $\mathbf{x}_i$.
2. Générer une nouvelle solution par vol de Lévy.
3. Choisir un nid j aléatoirement.
4. Si la nouvelle solution est meilleure, remplacer la solution du nid j .
5. Abandonner une fraction p_a des pires nids, créer de nouveaux nids aléatoirement.
6. Garder la meilleure solution.
7. Répéter jusqu'à convergence.

Formules mathématiques importantes

Génération par vol de Lévy

$$\mathbf{x}_i^{\text{new}} = \mathbf{x}_i + \alpha \oplus \text{Lévy}(\lambda)$$

Caractéristiques

Espace de recherche

- Optimisation continue
- Adaptable à d'autres types

Convergence

- Convergence généralement rapide
- Bonne exploration grâce aux vols de Lévy

Guided parameters

- p_a : Probabilité d'abandon (typiquement 0.25)
- α : Échelle du pas (généralement 1)
- λ : Paramètre d'échelle du vol de Lévy (typiquement 1.5)

Informations clés

- Inspiré du parasitisme de couvée des coucous
- Combinaison de vols de Lévy (exploration) et remplacement des pires solutions (exploitation)
- Chaque nid contient une solution
- Les "œufs" représentent de nouvelles solutions candidates
- Mécanisme d'abandon évite les minima locaux
- Simple avec peu de paramètres
- Efficace pour problèmes multimodaux
- Peut être combiné avec d'autres opérateurs de recherche

i Firefly Algorithm

Algorithme

1. Initialiser population aléatoire
2. Pour chaque paire (i,j), si $I_i < I_j$, déplacer i vers j
3. Mettre à jour intensités
4. Répéter

Formules

$$\beta = \beta_0 \exp(-\gamma r_{ij}^2)$$

$$\mathbf{x}_i = \mathbf{x}_i + \beta(\mathbf{x}_j - \mathbf{x}_i) + \alpha \epsilon$$

Caractéristiques

- Espace : Continu (extensions discrètes)
 - Convergence : Bonne, équilibre via γ
 - Guided parameters : β_0, γ, α
-

Cuckoo Search

Algorithme

1. Initialiser n nids aléatoires
2. Générer nouvelle solution par Lévy flight
3. Remplacer si meilleure
4. Abandonner fraction p_a des pires nids
5. Répéter

Formules

$$\mathbf{x}_i^{\text{new}} = \mathbf{x}_i + \alpha \oplus \text{Lévy}(\lambda)$$

Caractéristiques

- Espace : Continu
- Convergence : Rapide, bonne exploration
- Guided parameters : p_a, α, λ

Algorithmes Évolutionnaires

Algorithme

Étapes Succintes

1. **Initialisation** : génération aléatoire d'une population de n solutions candidates

2. **Évaluation** : calcul de la fitness pour chaque individu
 3. **Sélection** : choix des parents basé sur leur fitness
 4. **Croisement** : recombinaison des parents pour produire une descendance
 5. **Mutation** : modification aléatoire des individus
 6. **Remplacement** : formation de la nouvelle génération avec élitisme
 7. **Répétition** des étapes 2-6 jusqu'à critère d'arrêt
-

Formules Mathématiques Importantes

Sélection Proportionnelle (Roulette Wheel)

$$p_i = \frac{f_i}{\sum_{j=1}^n f_j}$$

où p_i = probabilité de sélection de l'individu i , f_i = sa fitness

Sélection par Rang Linéaire

$$p_i = \frac{1}{n} \left[\beta - 2(\beta - 1) \frac{i - 1}{n - 1} \right]$$

avec $\beta \in [1, 2]$ contrôlant la pression de sélection, i = rang après tri

Théorème des Schémas (Holland)

$$M(t + 1, S) \geq \frac{M(t, S)f(S, t)}{F(t)} \left[1 - p_c \frac{\delta(S)}{m - 1} - o(S)p_m \right]$$

- $M(t, S)$: nombre d'individus correspondant au schéma S
- $\delta(S)$: longueur définissante du schéma
- $o(S)$: ordre du schéma (nombre de positions fixes)

Temps de Prise de Contrôle (Takeover Time)

$$\tau = \frac{\ln(n - 1)}{\alpha}$$

τ petit $\rightarrow$ sélection forte, τ grand $\rightarrow$ sélection faible

Caractéristiques

Espace de Recherche

- **Adapté à** : espaces discrets, continus, mixtes, permutations
- **Représentations** : binaire, réelle, permutation, arborescente (GP)
- **Fonction objectif** : boîte noire, pas besoin de dérivées

Convergence

- **Convergence globale** : évite les minima locaux grâce à la diversité

- **Diminishing returns** : amélioration rapide initiale, puis plateau
- **Probabiliste** : convergence en probabilité vers l'optimum global sous conditions

Guided Parameters

1. **Taille population (n)** : 20-200 (trade-off exploration/exploitation)
2. **Probabilité croisement (p_c)** : 0.6-0.9 (recombinaison)
3. **Probabilité mutation (p_m)** : 0.001-0.1 par gène (nouveauauté)
4. **Taux élitisme** : 1-10% (préservation meilleurs)
5. **Pression sélection (β ou k)** : contrôle vitesse convergence

Exploration vs Exploitation

- **Sélection** → exploitation (intensification)
 - **Mutation** → exploration (diversification)
 - **Croisement** → exploration modérée
 - **Équilibre critique** contrôlé par p_c, p_m, n
-

Informations Clés

Variantes Principales

- **Sélection** : proportionnelle, par rang, tournoi ($k = 2$ typique)
- **Croisement** : 1-point, 2-point, uniforme, spécifique au problème
- **Mutation** : bit-flip, gaussienne, swap, insertion

Avantages

- Pas besoin d'information de gradient
- Robustesse aux bruits et discontinuités
- Exploration globale efficace
- Parallélisation naturelle

Limitations

- Nombreux paramètres à régler
- Convergence parfois lente en fin d'optimisation
- Coût calcul élevé pour fonctions objectif complexes

Bonnes Pratiques

- **Encodage réel** recommandé pour optimisation continue
- **Élitisme** essentiel pour convergence monotone
- **Diversité** à surveiller pour éviter convergence prématurée
- **Paramètres adaptatifs** possibles pour améliorer performances

Applications Typiques

- Optimisation combinatoire (TSP, emploi du temps)
- Optimisation continue (ingénierie, calibration)
- Apprentissage automatique (réglage hyperparamètres)

- Conception (aérodynamique, circuits)

Algorithme

1. Initialiser population aléatoire
 2. Évaluer fitness de chaque individu
 3. Sélectionner les parents (par roulette, rang ou tournoi)
 4. Appliquer croisement avec probabilité p_c
 5. Appliquer mutation avec probabilité p_m
 6. Former nouvelle génération avec élitisme
 7. Répéter 2-6 jusqu'à critère d'arrêt
-

Formules Mathématiques

Sélection proportionnelle

$$p_i = \frac{f_i}{\sum_{j=1}^n f_j}$$

Théorème des schémas

$$M(t+1, S) \geq \frac{M(t, S)f(S, t)}{F(t)} \left[1 - p_c \frac{\delta(S)}{m-1} - o(S)p_m \right]$$

Sélection par rang linéaire

$$p_i = \frac{1}{n} \left[\beta - 2(\beta - 1) \frac{i-1}{n-1} \right]$$

Caractéristiques

Espace de recherche

- Discret, continu, permutations
- Encodage : binaire, réel, permutation
- Fonction boîte noire (pas de gradient)

Convergence

- Convergence globale (évite minima locaux)
- Diminishing returns (plateau en fin)
- Probabiliste

Guided Parameters

- n : taille population (20-200)
- p_c : probabilité croisement (0.6-0.9)
- p_m : probabilité mutation (0.001-0.1)
- Élitisme : 1-10%
- Pression sélection (β ou k tournoi)

Programmation Génétique (GP)

Algorithme

1. **Initialisation** : Générer une population initiale de programmes aléatoires à partir des ensembles de fonctions F et de terminaux T
2. **Évaluation** : Calculer la fitness de chaque programme selon une métrique définie (ex: erreur quadratique)
3. **Sélection** : Choisir les programmes les plus performants pour la reproduction
4. **Recombinaison** : Appliquer les opérateurs génétiques :
 - **Crossover** : Échanger des sous-arbres entre deux programmes parents
 - **Mutation** : Remplacer un sous-arbre aléatoire par un nouveau
5. **Remplacement** : Former une nouvelle génération à partir des descendants et/ou des parents
6. **Terminaison** : Répéter jusqu'à satisfaction d'un critère (fitness, générations, stagnation)

Formules Mathématiques

Fitness pour régression symbolique :

$$f(p) = \sum_{i=1}^k |y_i - p(x_i)|$$

où p est le programme, (x_i, y_i) sont les données d'entraînement.

Représentation arborescente :

Un programme p représenté comme arbre, où :

- Nœuds internes : fonctions de F
- Feuilles : terminaux de T

Caractéristiques

Espace de recherche :

- Toutes les compositions possibles de F et T
- Potentiellement infini et de dimension variable
- Contraint par la propriété de clôture

Convergence :

- Convergence probable vers des solutions sous-optimales
- Sensible au surapprentissage (overfitting)
- Nécessité de limiter la complexité (contrôle du "bloat")

Guided Parameters :

- **Fonction set (F)** : Opérateurs autorisés (+, -, ×, ÷, fonctions booléennes, etc.)
- **Terminal set (T)** : Variables et constantes

- **Taille population** : 50-1000 individus
- **Probabilité crossover** : 80-95%
- **Probabilité mutation** : 5-20%
- **Méthode sélection** : Tournoi, roulette, rang
- **Limite profondeur** : 5-20 niveaux
- **Critère arrêt** : Générations (50-500) ou fitness cible

Informations Clés

Avantages :

- Pas de forme fonctionnelle pré-définie
- Solutions interprétables (formules explicites)
- Construction automatique de caractéristiques
- Exploration non-linéaire naturelle

Défis :

- Complexité computationnelle élevée
- Contrôle de la taille (bloat problem)
- Risque de surapprentissage
- Difficulté avec grands jeux de données

Applications typiques :

- Régression symbolique
- Classification
- Modélisation de comportements (robotique)
- Découverte de lois scientifiques

Variantes :

- **GP arborescent** : Représentation en arbre, opérateurs sur sous-arbres
- **GP linéaire** : Programmes séquentiels, stack-based
- **Grammatical Evolution** : Génération via grammaire formelle

Algorithme

1. **Initialiser** population de programmes aléatoires via F et T
2. **Évaluer** chaque programme avec fonction fitness (ex: erreur absolue)
3. **Sélectionner** meilleurs individus (tournoi/roulette)
4. **Appliquer** opérateurs :
 - Crossover : échanger sous-arbres entre parents
 - Mutation : remplacer sous-arbre aléatoire
5. **Remplacer** ancienne génération
6. **Répéter** 2-5 jusqu'à critère d'arrêt

Formules

Fitness régression :

$$f(p) = \sum_{i=1}^k |y_i - p(x_i)|$$

Caractéristiques

Espace : Composition infinie de F et T (arbres variables)

Convergence : Vers optimum local, sensible au surapprentissage et bloat

Paramètres :

- F (fonctions), T (terminaux)
- Taille population (50-1000)
- Probabilités crossover (0.8-0.95) et mutation (0.05-0.2)
- Méthode sélection (tournoi)
- Limite profondeur (5-20)
- Critère arrêt (générations/stagnation)

Performance of Metaheuristics

Challenges

- **Stochasticité et Variance** : Les exécutions multiples donnent des résultats différents.
- **Absence de Métrique Universelle** : Pas d'indicateur unique pour mesurer la performance.
- **Dépendance aux Paramètres** : La performance est sensible aux *guided parameters*.
- **Dépendance au Problème** : Une méthode performante sur un problème peut être inefficace sur un autre.
- **Théorème “No Free Lunch” (NFL)** : Aucun algorithme n'est supérieur à un autre sur l'ensemble de tous les problèmes.

Key Mathematical Formulas

1. Mesure de l'Effort de Calcul (Proxy)

Pour un problème de taille N , l'effort T (ex: nombre d'évaluations) peut suivre une loi de puissance :

$$T(N) = C \times N^\alpha$$

où C et α sont des constantes dépendant de l'algorithme et du problème.

2. Taux de Succès (Probability of Success)

Probabilité de trouver une solution dans une marge d'erreur ϵ sur $\mathcal{N}$ runs :

$$P_\epsilon = \frac{\text{Nombre de solutions avec erreur} \leq \epsilon}{\mathcal{N}}$$

3. Théorème "No Free Lunch" (Formulation)

Pour deux algorithmes A_1 et A_2 , sur l'ensemble F de toutes les fonctions discrètes possibles :

$$\sum_{f \in F} P(d_m \mid f, m, A_1) = \sum_{f \in F} P(d_m \mid f, m, A_2)$$

où $P(d_m \mid f, m, A)$ est la probabilité que la séquence de m échantillons d_m générée par A contienne l'optimum de f .

Performance Evaluation

Métriques Principales

- **Effort de Calcul** : Temps machine ou nombre d'évaluations de la fonction objectif.
- **Qualité de la Solution** : Erreur relative moyenne ou taux de succès P_ϵ .
- **Évolutivité (Scalability)** : Croissance de l'effort en fonction de la taille du problème N .

Protocole Expérimental

- Les résultats doivent être **moyennés** sur de nombreux runs (typiquement $\mathcal{N} \in [100, 1000]$).
- Les comparaisons doivent utiliser des **tests statistiques** (ex: Kolmogorov-Smirnov) pour valider la significativité.
- Les *benchmarks* sont essentiels (problèmes réels ou synthétiques comme les NK -landscapes).

Informations Clés

Nature de l'Analyse

L'analyse de performance est **empirique et statistique**, non théorique, en raison du caractère stochastique des métaheuristiques.

Guided Parameters Essentiels (Exemples)

Chaque métaheuristique a des paramètres critiques à régler :

- **Recuit Simulé** : Température initiale, calendrier de refroidissement.

- **Algorithme Génétique** : Taille de population, taux de croisement/mutation.
- **Optimisation par Essaims Particulaires (PSO)** : Coefficients cognitif et social, poids d'inertie.

Conséquence Pratique du Théorème NFL

Il n'existe pas de méthode universellement meilleure. Le choix et le réglage d'une métaheuristique doivent être **adaptés à la classe de problème spécifique** visée. La recherche d'une méthode "générale" est vaine.

Challenges

- Stochasticité $\rightarrow$ résultats variables
- Pas de métrique universelle
- Dépendant des paramètres
- Dépendant du problème
- “No Free Lunch” : aucun algorithme n’est meilleur sur tous les problèmes

Formules Clés

Effort de calcul :

$$T(N) \propto N^\alpha$$

Taux de succès :

$$P_\epsilon = \frac{\text{solutions dans } \epsilon}{\text{runs}}$$

Théorème NFL :

$$\sum_f P(d_m \mid f, m, A_1) = \sum_f P(d_m \mid f, m, A_2)$$

Évaluation

Métriques :

- Effort : temps ou évaluations
- Qualité : erreur relative ou P_ϵ
- Évolutivité : croissance avec N

Protocole :

- Moyenne sur 100-1000 runs
- Tests statistiques pour comparaison
- Benchmarks réels et synthétiques

Informations Essentielles

- Analyse empirique et statistique
- Paramètres critiques à régler
- Aucune méthode universelle
- Adapter l’algorithme au problème

Transitions de phase en optimisation

Challenges

- **Transition abrupte** de complexité : passage de linéaire à exponentiel
- **Raréfaction des solutions** dans l'espace de recherche
- **Dépendance critique** au paramètre de difficulté $\alpha = M/N$
- **Comportement non monotone** des métaheuristiques
- **Seuils critiques** différents selon l'algorithme utilisé

Formules mathématiques importantes

Définition du problème

- Énergie : E = nombre de contraintes non satisfaites
- Paramètre de contrôle : $\alpha = \frac{M}{N}$

Seuils critiques

- **2-XORSAT** : $\alpha_c = \frac{1}{2}$
- **3-XORSAT** :
 - $\alpha_d = 0.8184$ (début du clustering)
 - $\alpha_c = 0.9179$ (transition de phase)
- **RWSAT sur 3-XORSAT** : $\alpha_E = 0.33$
- **Backtracking** : $\alpha_c = \frac{2}{3}$ (assignation aléatoire)

Complexité algorithmique

- Régime linéaire : $\langle T_{res} \rangle = N \cdot t_{res}$
- Régime exponentiel : $\langle T_{res} \rangle = \exp(N \cdot \tau_{res})$

Performance evaluation

Métriques principales

- **Probabilité de satisfaisabilité** : $P_{SAT}(N, \alpha)$
- **Temps de résolution moyen** : $\langle T_{res} \rangle$
- **Énergie moyenne** : $e = E/M$
- **Valeur de plateau** : e_{plateau}

Comportements observés

- **Régime facile** ($\alpha < \alpha_E$) :
 - $e_{\text{plateau}} = 0$
 - Complexité linéaire en N
 - Convergence rapide
- **Régime difficile** ($\alpha_E \leq \alpha < \alpha_c$) :
 - $e_{\text{plateau}} > 0$
 - Complexité exponentielle en N
 - Fluctuations autour d'un plateau

- **Régime insoluble ($\alpha > \alpha_c$) :**
 - $P_{SAT} \rightarrow 0$ pour $N \rightarrow \infty$
 - Aucune solution avec haute probabilité

Informations clés

Structure de l'espace des solutions

- $\alpha < \alpha_d$: solutions uniformément distribuées
- $\alpha_d \leq \alpha < \alpha_c$: solutions groupées en clusters
- $\alpha > \alpha_c$: absence de solutions (haute probabilité)

Algorithmes et leurs caractéristiques

Gauss elimination

- Complexité : $\mathcal{O}(N^3)$ pour XORSAT
- Toujours polynomial pour XORSAT

RWSAT (Random Walk SAT)

- Heuristique locale
- Transition à $\alpha_E = 0.33$ pour 3-XORSAT
- Sensible à l'initialisation aléatoire

Backtracking

- Exploration systématique en profondeur
- Transition à $\alpha_c = 2/3$ (version aléatoire)
- Meilleure avec heuristique de choix de variable ($\alpha_c = 0.7507$)

Implications pratiques

1. **Identification du régime** : déterminer si α est inférieur ou supérieur au seuil critique
2. **Choix d'algorithme** : adapter la méthode au régime de difficulté
3. **Dimensionnement** : relation entre N , M et la difficulté attendue
4. **Attentes de performance** : prévoir le temps de calcul en fonction de α

Guide des paramètres

- **Contrôle principal** : $\alpha = M/N$
- **Seuils à surveiller** : $\alpha_E, \alpha_d, \alpha_c$
- **Indicateur de performance** : $e = E/M$
- **Critère d'arrêt** : $E = 0$ ou temps maximal

Recommandations

- Pour $\alpha < \alpha_E$: méthodes simples suffisent (complexité linéaire)
- Pour $\alpha_E \leq \alpha < \alpha_c$: prévoir des méthodes robustes (complexité exponentielle)
- Pour $\alpha > \alpha_c$: problème probablement insoluble, considérer la relaxation des contraintes

Challenges

- Complexité passe brutalement de linéaire à exponentielle
- Solutions deviennent rares quand $\alpha = M/N$ augmente
- Performances dépendent fortement du régime (α)

Formules clés

- Paramètre : $\alpha = M/N$
- 2-XORSAT : $\alpha_c = 0.5$
- 3-XORSAT : $\alpha_c = 0.9179$
- RWSAT : transition à $\alpha_E = 0.33$

Performance

Métriques

- P_{SAT} : probabilité de satisfaisabilité
- $\langle T_{res} \rangle$: temps moyen de résolution
- $e = E/M$: énergie normalisée

Régimes

- **Linéaire** ($\alpha < \alpha_E$) : $T \sim N$
- **Exponentiel** ($\alpha_E \leq \alpha < \alpha_c$) : $T \sim \exp(N)$
- **Insoluble** ($\alpha > \alpha_c$) : $P_{SAT} \rightarrow 0$

Informations essentielles

1. **Seuils critiques :**
 - 2-XORSAT : 0.5
 - 3-XORSAT : 0.9179
 - RWSAT : 0.33
 - Backtracking : 0.667
2. **Structure solutions :**
 - $\alpha < 0.818$: solutions uniformes
 - $\alpha \in [0.818, 0.918]$: solutions en clusters
 - $\alpha > 0.918$: aucune solution
3. **Recommandations :**
 - Monitorer α
 - Changer d'algorithme selon le régime
 - Anticiper explosion exponentielle